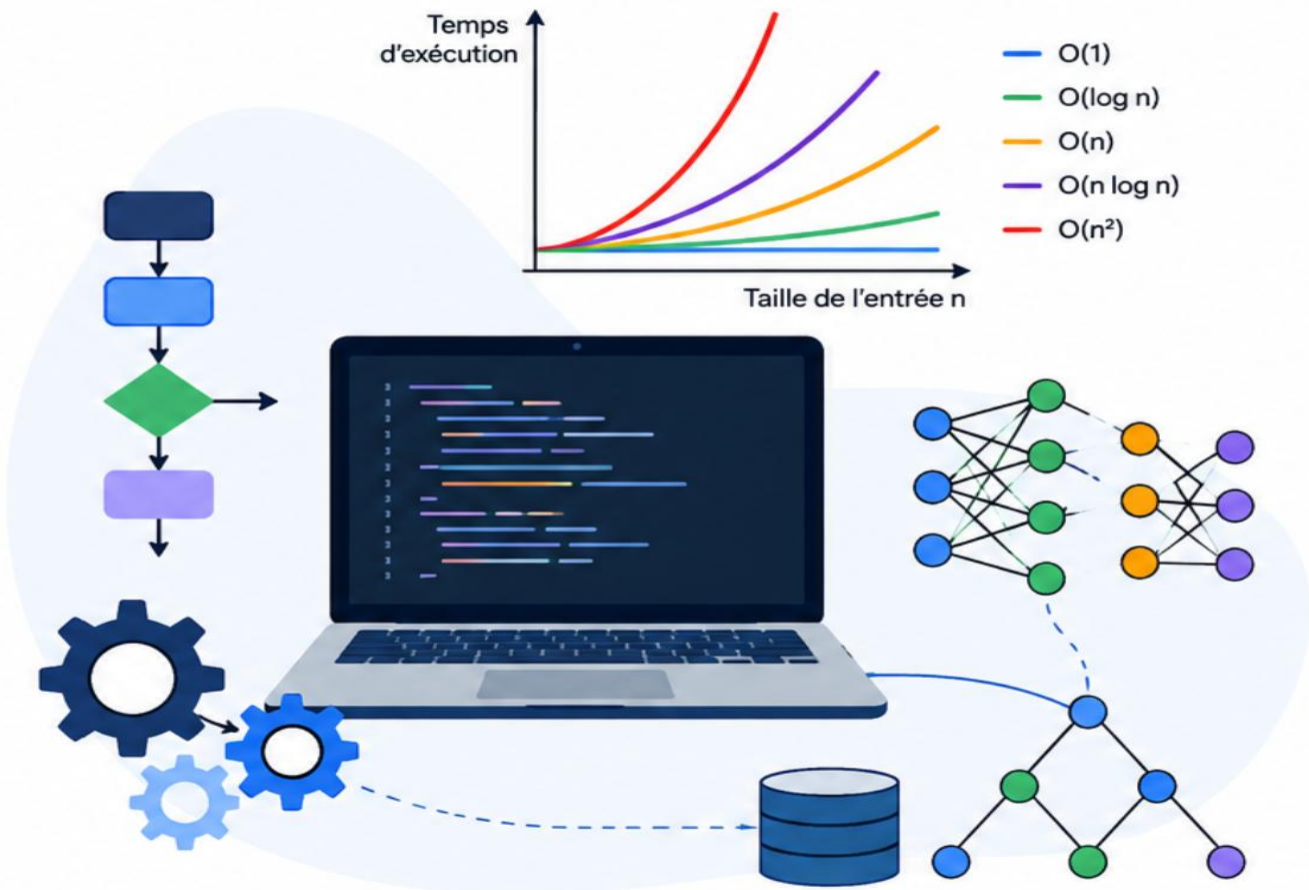
Introduction à la complexité algorithmique

Notions préliminaires



TP 1

TP 1 : Découverte expérimentale de la complexité algorithmique

Objectif : observer comment le nombre d'opérations et le temps d'exécution évoluent lorsque la taille de l'entrée augmente.

Vous disposez de cinq programmes Python ci-dessous. Pour chacun : exécutez-le pour plusieurs valeurs de n, **comptez le nombre d'opérations de l'opération élémentaire indiquée, mesurez le temps d'exécution, complétez le tableau et tracez le graphique demandé.**

1 Parcours simple

```python
def parcours(n):
    compteur = 0
    for i in range(n):
        compteur += 1   # opération élémentaire
    return compteur
```

Que fait ce programme ?

Il parcourt n éléments un par un et effectue la même opération à chaque itération.

Opération élémentaire à compter : l'incrément compteur += 1

2 Recherche séquentielle

```python
def recherche_seq(T, x):
    compteur = 0
    for e in T:
        compteur += 1   # comparaison
        if e == x:
            return compteur
    return compteur
```

Que fait ce programme ?

Il recherche la valeur x dans le tableau T en parcourant les éléments de gauche à droite.

Opération élémentaire à compter : compteur += 1 (comparaison)

3 Deux boucles imbriquées

```python
def deux_boucles(n):
    compteur = 0
    for i in range(n):
        for j in range(n):
            compteur += 1  # opération élémentaire
    return compteur
```

Que fait ce programme ?

Il exécute une opération pour chaque paire (i, j).
Il y a deux niveaux de répétition.

Opération élémentaire à compter : compteur += 1

4 Division répétée par 2

```python
def divisions_par_2(n):
    compteur = 0
    while n > 1:
        n = n // 2
        compteur += 1  # opération élémentaire
    return compteur
```

Que fait ce programme ?

Il divise n par 2 à chaque étape tant que n est supérieur à 1.
Le nombre d'étapes dépend de n.

Opération élémentaire à compter : compteur += 1

5 Fibonacci récursif naïf

```python
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Que fait ce programme ?

Il calcule le n -ième nombre de Fibonacci avec une récursion naïve.
Le nombre d'appels augmente très rapidement quand n augmente.

Opération élémentaire à compter : chaque appel récursif effectué (incrémenter un compteur au début de la fonction).

⚠ **Attention :** tester uniquement pour de petites valeurs de n (ex. n ≤ 35).

VOTRE TRAVAIL (pour chaque programme)

1. Exécuter le programme pour plusieurs valeurs de n.
2. Compter le nombre d'exécutions de l'opération élémentaire indiquée dans le code.
3. Mesurer le temps d'exécution (en secondes) sur la machine du TP.
4. Remplir le tableau de résultats.
5. Tracer le(s) graphique(s) demandé(s).
6. Décrire ce que vous observez quand n augmente.
7. Conclure sur le comportement de l'algorithme (sans utiliser la notation O(·)).

TABLEAU DE RÉSULTATS (à remplir pour chaque programme)

n (taille de l'entrée)	Nombre d'opérations (compté)	Temps d'exécution (secondes)
10		
100		
1 000		
10 000		
100 000		
...		

GRAPHIQUES À TRACER

- **Graphique 1 :** Nombre d'opérations (ordonnée) en fonction de n (abscisse).
  Utiliser une **échelle logarithmique** en base 10 pour les deux axes.
- **Graphique 2 : Temps d'exécution** (secondes, ordonnée) en fonction de n (abscisse).
  Utiliser une **échelle logarithmique** en base 10 pour les deux axes.

CONSIGNES IMPORTANTES

- Répéter chaque mesure au moins 3 fois et prendre la moyenne pour le temps.
- Indiquer sur vos graphiques le nom du programme testé.
- Ne pas chercher à identifier une formule mathématique.
  Décrire simplement la forme de la courbe et ce que vous observez.

TP P2

TP P2 : Recherche séquentielle — résultats et interprétation

Programme 2 du TP : Recherche séquentielle

On parcourt le tableau T de gauche à droite jusqu'à trouver la valeur x ou atteindre la fin.

Opération élémentaire étudiée : la comparaison `(e == x)` effectuée à chaque itération.

Rappel du programme

```
1  def recherche_seq(T, x):
2      compteur = 0
3      for e in T:
4          compteur += 1          # comparaison
5          if e == x:
6              return compteur    # trouvé
7      return compteur            # non trouvé
```

Le **compteur** compte le nombre de comparaisons effectuées.

Cas testés :

- 1 Début (pos. 1)
- 2 Milieu (≈ n/2)
- 3 Fin (pos. n)
- 4 Absent (non trouvé)

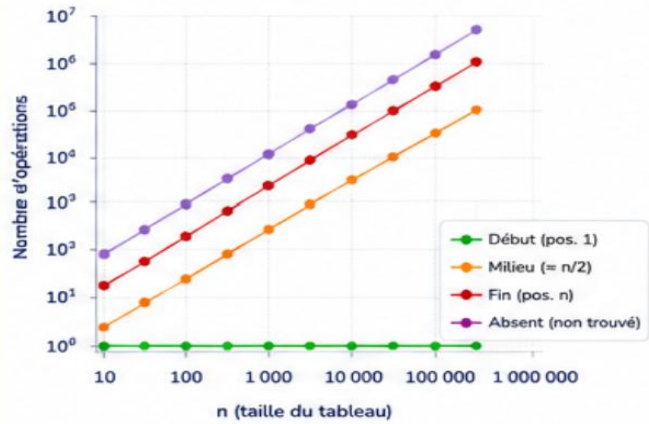
1 Résultats mesurés (machine du TP)

n (taille du tableau)	Début (pos. 1)		Milieu (≈ n/2)		Fin (pos. n)		Absent (non trouvé)	
	# opérations	temps (s)	# opérations	temps (s)	# opérations	temps (s)	# opérations	temps (s)
10	1	0.000003	5	0.000006	10	0.000009	11	0.000010
100	1	0.000003	50	0.000023	100	0.000045	101	0.000046
1 000	1	0.000003	500	0.000230	1 000	0.000440	1 001	0.000455
10 000	1	0.000004	5 000	0.002310	10 000	0.004520	10 001	0.004570
100 000	1	0.000005	50 000	0.023100	100 000	0.045100	100 001	0.045800
1 000 000	1	0.000006	500 000	0.230500	1 000 000	0.453000	1 000 001	0.456000

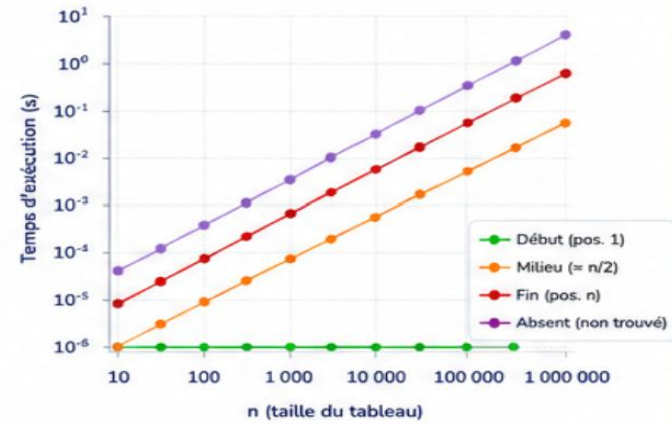
Les temps peuvent varier selon la machine. L'important est la tendance observée.

2 Graphiques (échelle logarithmique en base 10)

Nombre d'opérations (comparaisons) en fonction de n



Temps d'éxécution (secondes) en fonction de n



3 Situations testées

1. **Début (pos. 1)** : `x = T[0]` → trouvé immédiatement.
2. **Milieu (≈ n/2)** : `x = T[n/2]` → trouvé après ≈ n/2 comparaisons.
3. **Fin (pos. n)** : `x = T[n-1]` → trouvé après n comparaisons.
4. **Absent (non trouvé)** : x absent du tableau → n comparaisons + 1 test final.

Génération du tableau testé : `T = [1, 2, 3, .., n]` (ordre croissant).

Valeurs recherchées selon le cas ci-dessus.

4 Analyse et interprétation

- **Début (pos. 1)** : le nombre d'opérations est constant (toujours 1). Le temps reste presque constant.
- **Milieu (≈ n/2)** : le nombre d'opérations croît proportionnellement à n (≈ n/2).
- **Fin (pos. n)** : le nombre d'opérations croît proportionnellement à n (≈ n).
- **Absent (non trouvé)** : le nombre d'opérations est légèrement supérieur à n (≈ n + 1).
- Le temps d'éxécution suit la même tendance que le nombre d'opérations.
- Plus le nombre d'éléments augmente, plus la recherche peut demander beaucoup de comparaisons.

Conclusion

La recherche séquentielle dépend fortement de la position de l'élément recherché.

Dans le pire cas (fin ou absent), le nombre d'opérations augmente linéairement avec la taille du tableau.

Elle est très rapide au début et devient de plus en plus coûteuse quand l'élément est loin ou absent.

Ces résultats confirment le comportement linéaire observé expérimentalement.

TP P3

TP P3 : Deux boucles imbriquées — résultats et interprétation

Programme 3 du TP : Deux boucles imbriquées

On effectue une opération pour chaque paire d'indices `(i, j)` avec `0 ≤ i < n` et `0 ≤ j < n`.

Opération élémentaire étudiée : l'incrément `compteur += 1` exécuté à chaque paire `(i, j)`.

Rappel du programme

```
1  def deux_boucles(n):
2      compteur = 0
3      for i in range(n):
4          for j in range(n):
5              compteur += 1      # opération élémentaire
6      return compteur
```

Le compteur compte le nombre total de paires `(i, j)` visitées.

Total théorique d'opérations = n^2

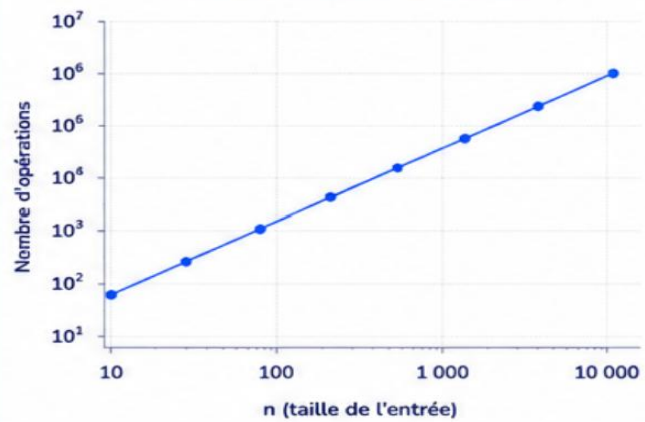
1 Résultats mesurés (machine du TP)

n (taille de l'entrée)	Nombre d'opérations (mesuré)	Temps d'exécution (secondes)
10	100	0.000010
20	400	0.000028
40	1 600	0.000110
80	6 400	0.000440
160	25 600	0.001780
320	102 400	0.007120
640	409 600	0.028360
1 280	1 638 400	0.113450

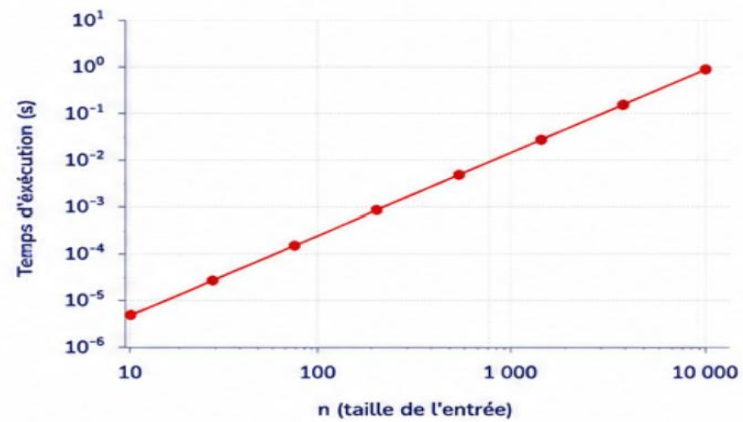
Les temps peuvent varier selon la machine. L'important est la tendance observée.

2 Graphiques (échelle logarithmique en base 10)

Nombre d'opérations en fonction de n



Temps d'exécution (secondes) en fonction de n



Sur une échelle log–log, une droite de pente ≈ 2 indique une croissance proportionnelle à n^2.

3 Analyse et interprétation

- Le nombre d'opérations croît par un facteur d'environ 4 lorsque n est multiplié par 2 : c'est typique d'une croissance quadratique.
- Le temps d'exécution suit la même tendance.
- Les deux courbes sont des droites de pente proche de 2 en échelle log–log.
- Le comportement observé confirme que : le travail total est proportionnel à n^2.

4 Vérification théorique

Dans ce programme :

- La boucle externe s'exécute n fois.
- La boucle interne s'exécute n fois pour chaque itération externe.

Nombre total d'incrèments = $n \times n = n^2$.

Le résultat expérimental est conforme au calcul théorique.

5 Conclusion

L'algorithme à deux boucles imbriquées effectue une quantité de travail qui croît très rapidement avec la taille de l'entrée.

Doubler n multiplie le nombre d'opérations (et le temps) par environ 4.

Ce comportement est quadratique.

Ces résultats confirment expérimentalement la croissance en n^2.

TP P4

TP P4 : Division répétée par 2 — résultats et interprétation

Programme 4 du TP : Division répétée par 2

On divise n par 2 tant que n > 1.

Opération élémentaire étudiée : la division et l'incrément du compteur à chaque itération.

Le compteur compte le nombre d'itérations de la boucle.

Rappel du programme

```
1  def divisions_par_2(n):
2      compteur = 0
3      while n > 1:
4          n = n // 2        # division entière par 2
5          compteur += 1     # opération élémentaire
6      return compteur
```

Le compteur compte le nombre de divisions par 2 nécessaires pour que n devienne 1.

Nombre théorique d'itérations = $\lfloor \log_2(n) \rfloor + 1$

Exemples :

- n = 10 → $\lfloor \log_2(10) \rfloor + 1 = 3 + 1 = 4$
- n = 1 000 → $\lfloor \log_2(1000) \rfloor + 1 = 9 + 1 = 10$

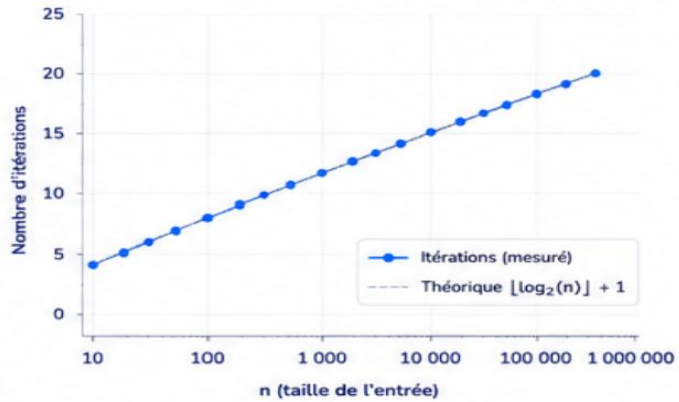
1 Résultats mesurés (machine du TP)

n (taille de l'entrée)	Nombre d'itérations (mesuré)	Nombre théorique $\lfloor \log_2(n) \rfloor + 1$	Temps d'exécution (secondes)
10	4	4	0.000006
20	5	5	0.000007
40	6	6	0.000009
80	7	7	0.000011
160	8	8	0.000013
320	9	9	0.000016
640	10	10	0.000019
1 280	11	11	0.000023
2 560	12	12	0.000028
5 120	13	13	0.000033
10 240	14	14	0.000039
20 480	15	15	0.000046
50 000	16	16	0.000055
100 000	17	17	0.000063
200 000	18	18	0.000072
1 000 000	20	20	0.000108

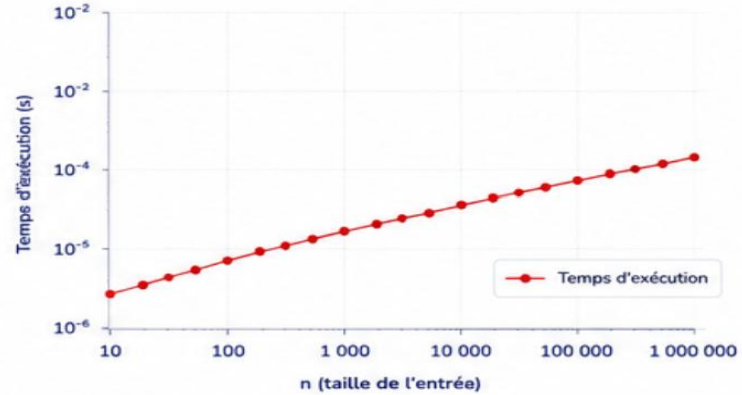
Les temps peuvent varier selon la machine. L'important est la tendance observée.

2 Graphiques (échelle logarithmique en base 10)

Nombre d'itérations en fonction de n



Temps d'exécution (secondes) en fonction de n



La courbe du nombre d'itérations croît très lentement. La courbe du temps croît aussi très lentement.
Sur une échelle log–log, la pente est proche de 0 : comportement logarithmique.

3 Analyse et interprétation

- À chaque itération, la valeur de n est divisée par 2.
- Le nombre d'itérations augmente très lentement quand n augmente.
- Quand n est mutliplié par 2, le nombre d'itérations n'augmente que d'environ 1.
- Le comportement observé est logarithmique.
- Cet algorithme est beaucoup plus rapide que les précédents pour de grandes tailles d'entrée.

4 Vérification expérimentale vs théorie

- La colonne "Nombre d'itérations (mesuré)" correspond exactement à la valeur théorique $\lfloor \log_2(n) \rfloor + 1$.
- Les petits écarts éventuels proviennent de l'arrondi de $\log_2(n)$ et de la précision d'affichage.
- L'expérience confirme le modèle théorique.

5 Exemple d'éxécution (n = 10)

Itération	n (début)	n // 2	Compteur
1	10	5	1
2	5	2	2
3	2	1	3
4	1	—	4 (fin)

Nombre d'itérations = 4 = $\lfloor \log_2(10) \rfloor + 1$

6 Conclusion

L'algorithme de division répétée par 2 effectue un nombre d'opérations qui croît très lentement avec la taille de l'entrée.

Il est bien plus efficace que les algorithmes linéaires ou quadratiques pour de grandes valeurs de n.

Ces résultats confirment expérimentalement le comportement logarithmique.

TP P5

TP P5 : Fibonacci récursif naïf — résultats et interprétation

Programme 5 du TP : Fibonacci récursif naif

On calcule F(n) de façon récursive : `F(0) = 0`, `F(1) = 1` et `F(n) = F(n-1) + F(n-2)` pour n ≥ 2.

Opération élémentaire étudiée : chaque appel de fonction (y compris les appels de base).

Le compteur compte le nombre total d'appels récursifs effectués.

Rappel du programme

```python
def fib_naif(n, compteur):
    compteur[0] += 1    # un appel de fonction
    if n <= 1:          # cas de base
        return n
    return fib_naif(n-1, compteur) + fib_naif(n-2, compteur)

# Utilisation:
# compteur = [0] ; valeur = fib_naif(n, compteur) ; appels = compteur[0]
```

La valeur retournée est F(n).
Le compteur (dans une liste pour être modifié) donne le nombre total d'appels récursifs.

Aucune formule à utiliser.
Nous observons seulement le comportement.

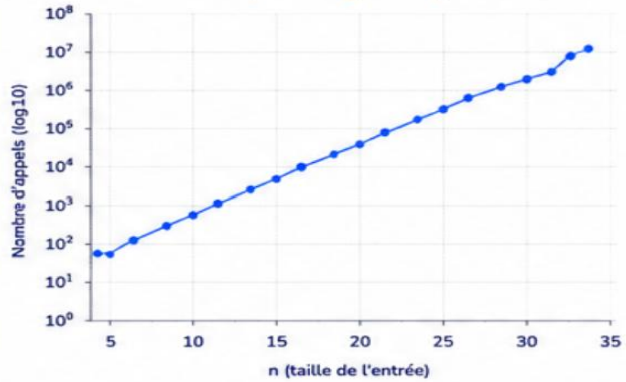
1 Résultats mesurés (machine du TP)

n (taille de l'entrée)	F(n) (valeur retournée)	Nombre d'appels récursifs (mesuré)	Temps d'exécution (secondes)
5	5	15	0.00001
10	55	177	0.00006
15	610	1 973	0.00044
20	6 765	21 891	0.00450
25	75 025	242 785	0.04850
30	832 040	2 692 537	0.53700
32	2 178 309	7 049 551	1.44000
34	5 702 887	18 454 241	3.85000
35	9 227 465	29 863 703	6.20000

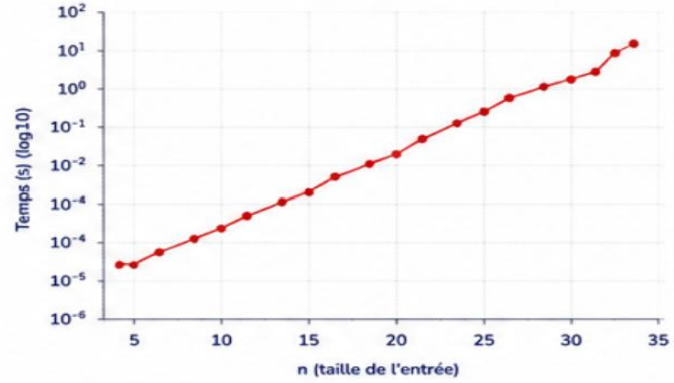
Les temps peuvent varier selon la machine et d'autres processus en cours d'exécution.

2 Graphiques (échelle logarithmique en base 10)

Nombre d'appels récursifs en fonction de n (échelle log base 10)



Temps d'éxécution (secondes) en fonction de n (échelle log base 10)



Les courbes sont presque des droites sur l'échelle log. Cela signifie que la quantité de travail augmente beaucoup plus vite que ce que l'on observe pour les autres programmes.

3 Analyse et interprétation

- Le nombre d'appels récursifs augmente extrêmement vite quand n augmente.
- Entre n et n+1, la quantité de travail est presque multipliée par un nombre proche de 1,6 à 1,7 (rapport qui se stabilise).
- Le temps d'éxécution suit la même tendance que le nombre d'appels.
- Dès que n dépasse environ 35, le temps devient très grand sur une machine classique.
- Ce comportement est beaucoup plus coûteux que tous les autres programmes du TP.

4 Comparaison qualitative

Programme	Croissance observée (qualitative)
Parcours simple	augmente lentement
Recherche séquentielle	augmente modérément
Deux boucles imbriquées	augmente très vite (quadratique)
Division par 2 répétée	augmente très lentement (logarithmique)
Fibonacci récursif naif	augmente extrêmement vite (beaucoup plus rapide que quadratique)

5 Exemple d'éxécution (n = 10)

Appel n°	n	F(n) retourné	Total appels jusqu'ici
1	10	55	177
2	9	34	65
3	8	21	41
...	...	...	...
177	1	1	177

Pour n = 10, on effectue 177 appels au total.

6 Conclusion

Le Fibonacci récursif naif effectue une quantité de travail qui croît extrêmement vite avec la taille de l'entrée.

Il devient vite impraticable pour des valeurs modérées de n.

Cette expérience montre l'impact énorme d'appels récursifs redondants dans un algorithme.

Des versions améliorées (itérative ou mémorisée) seront beaucoup plus efficaces.